



FOR JAVA DEVELOPERS

Checkout 24 s. CPU idle.

One @Transactional mistake in a Spring Boot flash sale: the row lock held through a payment call, measured, and the checkout that pays after the commit

Commit first. Then call out.

A flash sale: 20 orders and 200 product views a second. The checkout takes one unit of stock with a conditional UPDATE, then calls a card provider that answers in 100 ms, inside one @Transactional method. The row stays locked for the whole call. Checkout: 24.4 s. A different product page: 23.7 s. CPU: 29%. A pool of 50 changes nothing. Paying after the commit: 106 ms.

At the time of writing, October 2026: Spring Boot 4.1.1, Java 25, Postgres 17. One laptop, 2 CPUs per container, a stubbed provider: the shape of the failure, not a production benchmark.

This sheet covers

- An UPDATE holds its row lock until the commit
- Reading the stall: thread dump and pg_stat_activity
- Why a bigger pool only adds waiters on a hot row
- Reserve, commit, then pay with nothing held
- The cost: a charged order left RESERVED

Not covered here

- A reconciliation job: needed, and NOT built in the repository
- Declined cards and timeouts after the commit: untested
- Production capacity: one laptop and a stub provider

Class paths below are shortened to the part after src/main/java/dev/example/flashsale/. Every block is copied from the repository at commit b54cff1, and every output from evidence/.

01 Three setups, three runs each

Checkout, median	Pool 10 inside: 24.4 to 24.6 s. Pool 50 inside: 24.2 to 24.7 s. Pool 10 after commit: 106 to 107 ms
Browsing another product, median	Pool 10 inside: 23.4 to 23.7 s. Pool 50 inside: 20.4 to 20.5 s. After commit: 1 to 2 ms
Paid orders (1,200 asked for)	Pool 10 inside: 828 to 835. Pool 50 inside: 815 to 833. After commit: 1,201
Threads waiting for a connection, peak	Pool 10 inside: 187 to 188. Pool 50 inside: 148 to 149. After commit: 0
App CPU, peak	Pool 10 inside: 29 to 41%. Pool 50 inside: 28 to 31%. After commit: 27 to 39%
Five times the load, after commit	1,098 req/s, checkout median 103 ms, p99 141 ms, 6,001 of 6,001 paid, CPU 98%

Same sale every run: 20 orders and 200 product page views a second for 60 seconds, k6 constant-arrival-rate, the app on 2 CPUs. Ranges across three runs of each setup, from evidence/matrix.txt.



02 The checkout most of us write first

One transaction around the whole checkout: take one unit of stock, look up the price, charge the card, record the order. It reads safely. A declined card rolls the stock back, and so does a failed insert.

The tests pass, for both checkouts. They place one order at a time, so they cannot see a problem that only exists when two orders want the same row at once.

PayInsideTransaction.java

```
package dev.example.flashsale;

import java.util.UUID;

import org.springframework.boot.autoconfigure.condition.ConditionalOnProperty;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

/**
 * The version most of us write first: one transaction around the whole checkout,
 * including the call to the card provider.
 */
@Service
@ConditionalOnProperty(name = "checkout.mode", havingValue = "pay-inside",
    matchIfMissing = true)
public class PayInsideTransaction implements Checkout {

    private final ProductRepository products;
    private final OrderRepository orders;
    private final PaymentGateway payments;

    public PayInsideTransaction(ProductRepository products, OrderRepository orders,
        PaymentGateway payments) {
        this.products = products;
        this.orders = orders;
        this.payments = payments;
    }

    @Override
    @Transactional
    public OrderPlaced placeOrder(OrderRequest request) {
        if (!products.takeOne(request.productId())) {
            throw new SoldOut(request.productId());
        }
        int amount = products.price(request.productId());
        String ref = payments.charge(UUID.randomUUID().toString(), request.customer(),
            amount);
        long id = orders.insert(request.productId(), request.customer(), amount,
            "PAID");
        return new OrderPlaced(id, request.productId(), ref);
    }
}
```

```
evidence/tests.txt
```

```
PayAfterCommitTest
  ok    takesOneUnitAndRecordsOnePaidOrder
  ok    refusesWhenSoldOut
  ok    chargedButNotRecordedLeavesAReservedOrderToReconcile
  ok    isTheCheckoutInUse
PayInsideTransactionTest
  ok    takesOneUnitAndRecordsOnePaidOrder
  ok    refusesWhenSoldOut
  ok    isTheCheckoutInUse
Tests run: 7, Failures: 0
```

Seven tests, 0 failures. Both checkouts take one unit, record one PAID order, and refuse a sold out product.

Pinned by CheckoutContract, PayInsideTransactionTest and PayAfterCommitTest, 7 of 7 passing against Postgres 17 in Testcontainers: evidence/tests.txt.

03 The row lock

takeOne is the right way to take stock: the check and the write are one statement, so two buyers can never both take the last unit. But an UPDATE in Postgres takes a row lock, and holds it until the transaction ends.

In PayInsideTransaction the transaction ends after the card provider answers. So product 1 stays locked for the whole 100 ms payment call, and every other buyer's takeOne waits for it.

```
ProductRepository.java (excerpt: takeOne)
```

```
/** Takes one unit of stock. The row stays locked until the transaction ends. */
public boolean takeOne(long id) {
    return db.sql("""
        UPDATE products SET stock = stock - 1
        WHERE id = :id AND stock > 0""")
        .param("id", id)
        .update() == 1;
}
```

The row lock taken by takeOne is held until the transaction commits or rolls back.

PostgreSQL 17 documentation, [Explicit Locking](#) (linked below), and seen in every pay inside run: one session idle in transaction holding the row while the others wait on Lock:transactionid.



04 The sale: 24 seconds, CPU idle

Checkout takes 24.4 s at the median. A product page for an unrelated product takes 23.7 s. Nothing failed: it was all just slow. All 200 Tomcat threads were busy and the CPU peaked at 29%. The threads were not working. They were waiting.

At the midpoint, the thread dump and pg_stat_activity agree. Ten connections: nine sessions waiting on a lock (Lock:transactionid and Lock:tuple), and one idle in transaction, holding the row while its Java thread waits for the card provider. The other 189 threads wait to borrow a connection, browsers among them. The single active session with no wait is the snapshot query itself.

```
● ● ● evidence/runs/a-pool10-inside-1/summary.txt
```

```
pool=10 mode=pay-inside buy=20/s browse=200/s duration=60s 2026-10-09T07:05:44Z
order med 24.4s p99 48.4s max 51.2s failed 0.0%
browse med 23.7s p99 47.6s max 50.2s failed 0.0%
completed requests 9154 (101/s) never started (dropped) 216
paid orders in database 834
peak: connections in use 10.0 threads waiting for one 188.0
      tomcat busy threads 200.0 app cpu 29%
```

```
● ● ● evidence/runs/a-pool10-inside-1/threads.txt
```

```
tomcat request threads: 200
 189 waiting for a database connection
   9 waiting on Postgres
   1 waiting for the card provider
   1 taking this thread dump
```

```
● ● ● evidence/runs/a-pool10-inside-1/pg-waits.txt
```

```
Lock:transactionid active|8
Client:ClientRead idle in transaction|1
cpu:- active|1
Lock:tuple active|1
```

Pool 10, pay inside: checkout median 24.4 s, browse median 23.7 s, 834 paid orders, 200 busy Tomcat threads, app CPU 29% at the peak.

Measured, 2026-10-09: evidence/runs/<label>/summary.txt, threads.txt and pg-waits.txt in the repository, produced by bin/run.sh. One laptop, 2 CPUs per container, a stubbed provider.

05 The queue, measured

Twenty orders sent at the same instant, then one product page 50 ms later. Paying inside the transaction, the orders finish one after another, about 110 ms apart: 0.173 s, 0.281 s, 0.392 s and so on, the last at 2.305 s. The product page took 226 ms.

One order every 110 ms means at most about nine a second can finish, while twenty a second arrive. That is arithmetic on the measured spacing, not a separate measurement.

```
evidence/timeline-pay-inside.txt

kind    n  start_s  end_s  took_ms  status
order   0   0.000   0.616    616    200
order   1   0.000   0.849    849    200
order   2   0.000   1.848   1847    200
order   3   0.001   1.405   1404    200
order   4   0.001   1.074   1073    200
order   5   0.001   0.503    502    200
order   6   0.001   1.298   1297    200
order   7   0.001   0.727    726    200
order   8   0.001   2.305   2304    200
order   9   0.001   0.392    391    200
order  10   0.001   0.961    960    200
order  11   0.001   1.740   1739    200
order  12   0.001   2.183   2182    200
order  13   0.001   1.954   1953    200
order  14   0.001   1.516   1515    200
order  15   0.001   1.185   1183    200
order  16   0.001   2.071   2070    200
order  17   0.001   0.173    172    200
order  18   0.001   1.633   1631    200
order  19   0.001   0.281    279    200
browse  0   0.054   0.280    226    200
```

Paying inside: last of 20 orders at 2.305 s, product page 226 ms.

One burst, every request timed: `evidence/timeline-pay-inside.txt` and `timeline-pay-after-commit.txt`, produced by `bin/timeline.py`.

06 Five times the connections

Raise the Hikari pool from 10 to 50. Checkout: 24.2 s. Paid orders: 833. Browsing improves a little, to 20.4 s. Postgres now shows 48 sessions on Lock:transactionid and 1 on Lock:tuple, and still exactly one idle in transaction.

The extra connections did not add capacity. They gave more buyers a place to wait on the same row. A bigger pool helps workloads where waiting requests have independent work to do. It does not fix a hot row.

```
● ● ● evidence/runs/b-pool50-inside-1/summary.txt
```

```
pool=50 mode=pay-inside buy=20/s browse=200/s duration=60s 2026-10-09T07:07:41Z
order med 24.2s p99 49.5s max 55.1s failed 0.0%
browse med 20.4s p99 46.4s max 48.0s failed 0.0%
completed requests 9479 (105/s) never started (dropped) 18
paid orders in database 833
peak: connections in use 50.0 threads waiting for one 149.0
tomcat busy threads 200.0 app cpu 28%
```

```
● ● ● evidence/runs/b-pool50-inside-1/pg-waits.txt
```

```
Lock:transactionid active|48
Client:ClientRead idle in transaction|1
cpu:- active|1
Lock:tuple active|1
```

Pool 50, pay inside, three runs: checkout median 24.2 to 24.7 s, browse 20.4 to 20.5 s, 815 to 833 paid.

Nine runs on 2026-10-09, three per setup: evidence/matrix.txt, produced by bin/matrix.sh. Ranges across the three runs.

A bigger pool helps other workloads; it does not fix a hot row.

A design judgement, stated as such, not a measurement.



07 The fix: pay after the commit

A short transaction takes the stock and records the order RESERVED, then commits, releasing the row lock. The card provider is called with no connection and no lock held, with the idempotency key order-`<id>`. A second short transaction marks the order PAID.

There is no `@Transactional` on `placeOrder`. `TransactionTemplate` makes both boundaries visible in the method body; annotating the method would wrap the payment back into one long transaction.

PayAfterCommit.java

```
package dev.example.flashsale;

import org.springframework.boot.autoconfigure.condition.ConditionalOnProperty;
import org.springframework.stereotype.Service;
import org.springframework.transaction.support.TransactionTemplate;

/**
 * Reserve the stock and commit, then call the card provider with no
 * connection and no lock held, then record the payment in a second short transaction.
 */
@Service
@ConditionalOnProperty(name = "checkout.mode", havingValue = "pay-after-commit")
public class PayAfterCommit implements Checkout {

    private final ProductRepository products;
    private final OrderRepository orders;
    private final PaymentGateway payments;
    private final TransactionTemplate tx;

    public PayAfterCommit(ProductRepository products, OrderRepository orders,
                          PaymentGateway payments, TransactionTemplate tx) {
        this.products = products;
        this.orders = orders;
        this.payments = payments;
        this.tx = tx;
    }

    @Override
    public OrderPlaced placeOrder(OrderRequest request) {
        record Reserved(long orderId, int amount) {
        }
        Reserved reserved = tx.execute(status -> {
            if (!products.takeOne(request.productId())) {
                throw new SoldOut(request.productId());
            }
            int amount = products.price(request.productId());
            long orderId = orders.insert(request.productId(), request.customer(),
                                         amount, "RESERVED");
            return new Reserved(orderId, amount);
        });
        String ref = payments.charge("order-" + reserved.orderId(), request.customer(),
                                     reserved.amount());
        tx.executeWithoutResult(status -> orders.markPaid(reserved.orderId(), ref));
        return new OrderPlaced(reserved.orderId(), request.productId(), ref);
    }
}
```

```
evidence/runs/c-pool10-after-1/summary.txt
```

```
pool=10 mode=pay-after-commit buy=20/s browse=200/s duration=60s 2026-10-09T07:09:39Z
order med 106ms p99 126ms max 156ms failed 0.0%
browse med 1ms p99 9ms max 65ms failed 0.0%
completed requests 13202 (219/s) never started (dropped) 0
paid orders in database 1201
peak: connections in use 1.0 threads waiting for one 0.0
tomcat busy threads 5.0 app cpu 27%
```

```
evidence/runs/c-pool10-after-1/pg-waits.txt
```

```
Client:ClientRead idle|10
cpu:- active|1
```

```
evidence/timeline-pay-after-commit.txt
```

kind	n	start_s	end_s	took_ms	status
order	1	0.000	0.166	166	200
order	2	0.000	0.174	174	200
order	3	0.000	0.165	164	200
order	4	0.001	0.160	160	200
order	0	0.001	0.160	160	200
order	5	0.001	0.170	169	200
order	6	0.001	0.177	177	200
order	7	0.001	0.179	178	200
order	8	0.001	0.175	174	200
order	9	0.001	0.172	171	200
order	10	0.001	0.160	159	200
order	11	0.001	0.179	178	200
order	12	0.001	0.177	176	200
order	14	0.001	0.164	163	200
order	13	0.001	0.168	166	200
order	15	0.001	0.169	168	200
order	16	0.001	0.162	161	200
order	17	0.001	0.162	161	200
order	18	0.001	0.183	182	200
order	19	0.002	0.176	174	200
browse	0	0.052	0.067	15	200

Pay after commit, three runs: checkout median 106 to 107 ms, browse 1 to 2 ms, all 1,201 orders paid, no thread waiting for a connection.

Nine runs on 2026-10-09, three per setup: evidence/matrix.txt, produced by bin/matrix.sh. Ranges across the three runs.

A circular logo featuring a stylized cartoon character with glasses and a yellow background.

Burst: all 20 orders finished between 0.160 s and 0.183 s; the product page took 15 ms.

One burst, every request timed: `evidence/timeline-pay-inside.txt` and `timeline-pay-after-commit.txt`,
produced by `bin/timeline.py`.



08 What the fix costs

If the card is charged and markPaid then fails, the order stays RESERVED and the stock stays taken. The test below forces exactly that, and passes. The charge carried the key order-<id>, so it can be matched to the order.

Something has to find stuck RESERVED orders, ask the provider about key order-<id>, and either mark them PAID or release the stock. That reconciliation job is needed, and it is NOT built in the repository. Declined cards and provider timeouts after the first commit leave RESERVED orders too (read from the code, not tested).

The first version had this problem too, hidden in a longer transaction: a charge followed by a failed insert rolled back the order and kept the money, under a random UUID key nothing remembers. There is no transaction spanning your database and someone else's payment API.



```
src/test/java/dev/example/flashsale/PayAfterCommitTest.java
```

```
package dev.example.flashsale;

import static org.assertj.core.api.Assertions.assertThat;
import static org.assertj.core.api.Assertions.assertThatThrownBy;
import static org.mockito.ArgumentMatchers.anyLong;
import static org.mockito.ArgumentMatchers.anyString;
import static org.mockito.ArgumentMatchers.eq;
import static org.mockito.Mockito.doThrow;
import static org.mockito.Mockito.verify;

import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.dao.DataAccessResourceFailureException;
import org.springframework.test.context.bean.override.mockito.MockitoSpyBean;

@SpringBootTest(properties = "checkout.mode=pay-after-commit")
class PayAfterCommitTest extends CheckoutContract {

    @MockitoSpyBean OrderRepository orders;

    @Test
    void isTheCheckoutInUse() {
        assertThat(checkout).isInstanceOf(PayAfterCommit.class);
    }

    @Test
    void chargedButNotRecordedLeavesAReservedOrderToReconcile() {
        doThrow(new DataAccessResourceFailureException("database went away"))
            .when(orders).markPaid(anyLong(), anyString());

        var order = new OrderRequest(1, "Sarah Thompson");
        assertThatThrownBy(() -> checkout.placeOrder(order))
            .isInstanceOf(DataAccessResourceFailureException.class);

        long id = db.sql("SELECT id FROM orders").query(Long.class).single();
        assertThat(db.sql("SELECT status FROM orders WHERE id = :id").param("id", id)
            .query(String.class).single()).isEqualTo("RESERVED");
        assertThat(stock()).isEqualTo(2);
        verify(payments).charge(eq("order-" + id), eq("Sarah Thompson"), eq(2999));
    }
}
```

chargedButNotRecordedLeavesAReservedOrderToReconcile: status RESERVED, stock 2 of 3, charge sent with key order-<id> for 2999 cents.

Pinned by CheckoutContract, PayInsideTransactionTest and PayAfterCommitTest, 7 of 7 passing against Postgres 17 in Testcontainers: evidence/tests.txt.



The reconciler is not in the repository. Declined and timed out charges are read from the code, not tested.

Read from the code in the repository, not measured: `PayInsideTransaction.java`, `PayAfterCommit.java` and `PaymentGateway.java`.

09 Five times the load, and threads are not connections

The fixed checkout at 100 orders and 1,000 views a second, with 200 pre-allocated load users: 1,098 requests a second, checkout median 103 ms, p99 141 ms, all 6,001 paid, CPU 98%. Compute is the next limit. A run with 6,000 pre-allocated users at this rate failed with payment stub connect timeouts; the cause is not established and no conclusion is drawn from it.

Do 200 busy Tomcat threads mean refused connections? Holding 8,180 idle connections, a real request answered in 8 ms; at 8,200 it hung until a 10 s timeout. That is consistent with Tomcat's max-connections default of 8,192 at the time of writing. It ran through Docker Desktop's port proxy, so it was a hang rather than a refusal, and the layer is not claimed.

```
evidence/runs/cap-after-100/summary.txt
```

```
pool=10 mode=pay-after-commit buy=100/s browse=1000/s duration=60s 2026-10-09T07:55:51Z
order med 103ms p99 141ms max 221ms failed 0.0%
browse med 1ms p99 16ms max 73ms failed 0.0%
completed requests 66008 (1098/s) never started (dropped) 0
paid orders in database 6001
peak: connections in use 1.0 threads waiting for one 0.0
tomcat busy threads 13.0 app cpu 98%
```

```
evidence/connections.txt
```

```
idle connections held 8100 new connection refused 0 timed out 0 real request -> 200 in 61
ms
idle connections held 8180 new connection refused 0 timed out 0 real request -> 200 in 8 ms
idle connections held 8200 new connection refused 0 timed out 0 real request ->
TimeoutError in 10001 ms
idle connections held 8260 new connection refused 0 timed out 0 real request ->
TimeoutError in 10001 ms
idle connections held 8300 new connection refused 0 timed out 0 real request ->
TimeoutError in 10001 ms
```

Five times the load: 1,098 req/s, checkout median 103 ms, p99 141 ms, 6,001 paid, app CPU 98%.

Measured, 2026-10-09: evidence/runs/cap-after-100/summary.txt, produced by PRE_VUS=200 bin/run.sh cap-after-100 10 pay-after-commit 100 1000 60s.

8,180 idle connections held: real request 200 in 8 ms. 8,200 held: TimeoutError at 10 s.

Measured through Docker Desktop's port proxy: evidence/connections.txt, produced by bin/connections.py. The layer that held the connection is not established.

Tomcat max-connections 8,192, threads.max 200, accept-count 100.

Spring Boot 4.1.1 defaults at the time of writing, October 2026, as recorded in the repository README: Tomcat threads.max 200, accept-count 100, max-connections 8,192; HikariCP maximum-pool-size 10.



Before a transaction calls someone else's API

1. Never hold a row lock or a database connection while waiting on a network call
2. Remember an UPDATE holds its row lock until the transaction ends, not until the statement ends
3. Under load, read `pg_stat_activity`: Lock:transactionid plus one idle in transaction is a lock held across a slow call
4. All threads busy and the CPU idle means waiting, not working: take a thread dump before adding hardware
5. Do not raise the pool to fix a hot row; it adds waiters, not capacity
6. Test with an open model load (constant-arrival-rate), so the load does not slow down with the app
7. Reserve and commit, call the provider with nothing held, then record the result in a second short transaction
8. Use `TransactionTemplate` so the boundaries are visible, and keep `@Transactional` off the method that calls out
9. Give every charge an idempotency key derived from your own order id
10. Build the reconciliation job: stuck RESERVED orders, declined cards and timeouts all need it



Everything else

-  **Site**
<https://code-with-sam-dev.github.io>
-  **YouTube**
<https://www.youtube.com/@CodewithSam-Dev>
-  **LinkedIn**
<https://www.linkedin.com/company/code-with-sam-dev>
-  **x**
<https://x.com/CodeWithSamDev>
-  **TikTok**
<https://www.tiktok.com/@codewithsamdev>
-  **Instagram**
<https://www.instagram.com/codewithsamdev>
-  **Facebook**
<https://www.facebook.com/codewithsamdev>
-  **GitHub**
<https://github.com/code-with-sam-dev>
-  **Buy me a coffee**
<https://buymeacoffee.com/codewithsam>
-  **Buy me sadza**
<https://buymesadza.com/codewithsam>
- The repository: both checkouts, the tests, the load script, bin/run.sh and every evidence file**
<https://github.com/code-with-sam-dev/spring-boot-flash-sale>

PostgreSQL 17: explicit locking, row-level locks
<https://www.postgresql.org/docs/17/explicit-locking.html>

PostgreSQL 17: pg_stat_activity and wait events
<https://www.postgresql.org/docs/17/monitoring-stats.html>

Spring Framework: programmatic transaction management
<https://docs.spring.io/spring-framework/reference/data-access/transaction/programmatic.html>

k6: constant-arrival-rate executor
<https://grafana.com/docs/k6/latest/using-k6/scenarios/executors/constant-arrival-rate/>

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which system you want taken apart next.

Spring and Spring Boot are trademarks of Broadcom. Java is a trademark of Oracle. PostgreSQL is a trademark of the PostgreSQL Community Association of Canada. WireMock and k6 are trademarks of their respective owners. This is an independent, unofficial build produced by Code with Sam, not affiliated with or endorsed by any of them, and no third party artwork is reproduced here.

Technical claims verified 2026-10-09 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.